

Concurrent Echo server and client

In this example we describe a concurrent Echo server model.
That is, we are moving to a Concurrent Server model using fork().

This allows us to observe how the Kernel manages separate processes with isolated memory spaces and how File Descriptors are duplicated between parent and child.

Additionally, the inclusion of **Uppercase** logic adds a data manipulation step, demonstrating that the server is not merely a "pipe" but a processing unit.

Concurrent Echo Server: concurrent_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <ctype.h> // Required for the toupper function
#include <sys/wait.h> // Required for zombie process handling (waitpid)

#define PORT 8080
#define BUFFER_SIZE 4096

/**
 * Function to process the string - converts lowercase letters to uppercase.
 * Efficiency: Processing is done In-place on the Buffer to save memory allocations.
 */
void process_echo(char *str, ssize_t length) {
    for (ssize_t i = 0; i < length; i++) {
        str[i] = toupper((unsigned char)str[i]);
    }
}
```

```

int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    // 1. Create socket and set SO_REUSEADDR (as in the previous example)
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }

    // Increased backlog to 10 as the server is now concurrent
    if (listen(server_fd, 10) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Concurrent Server listening on port %d...\n", PORT);
}

```

```

while(1) {
    // 2. Accept - Waiting for a new client
    client_fd = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
    if (client_fd < 0) {
        perror("accept");
        continue;
    }

    printf("New client connected. Forking a new process...\n");

    /* * 3. The Magic of Concurrency: using fork()
     * fork() creates an exact duplicate of the server process.
     * Every process (Parent and Child) will have its own copy of the File Descriptors.
     */
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        close(client_fd);
    }

    else if (pid == 0) {
        /* --- Child Process Code --- */

        // The child does not need the listening socket; close it for efficiency
        close(server_fd);

        ssize_t bytes_read;
        while ((bytes_read = read(client_fd, buffer, BUFFER_SIZE)) > 0) {
            // Data processing: Conversion to Uppercase
            process_echo(buffer, bytes_read);

            // Send the processed data back to the client
            send(client_fd, buffer, bytes_read, 0);
        }

        printf("Child process (PID: %d) finished handling client.\n", getpid());
        close(client_fd);
        exit(0); // Terminate child process - very important!
    }
}

```

```

else {
    /* --- Parent Process Code --- */

    // The parent has finished its part with the current client; responsibility is passed to
    the child.
    // It is vital to close the FD here so no unnecessary Open Handles remain in the
    parent.
    close(client_fd);

    /* Zombie Process Cleanup:
    * Using waitpid with WNOHANG allows the parent to clean up finished children
    * without blocking (Non-blocking).
    */
    while (waitpid(-1, NULL, WNOHANG) > 0);
}
}

close(server_fd);
return 0;
}

```

1. Concurrency Mechanism

In the first example, the server was **Iterative**—if one client sent a large message slowly, all other clients were blocked. By using `fork()`:

- The moment an accept occurs, the server becomes a "process duplication factory."
- The **Parent** immediately returns to accept to receive the next client.
- The **Child** handles the current client completely independently.
- **Efficiency:** Full utilization of **Multi-core CPUs**—each process can run on a different core.

2. File Descriptor Management in Fork

This is a critical point: When a process splits, the Kernel increments the **Reference Count** of the File Descriptors.

- The **Parent** must close the `client_fd` because it is not using it. If it doesn't, the socket will never truly close (because the reference count will never reach zero).
- The **Child** must close the `server_fd` (the listening socket) for safety and efficiency, so it cannot accidentally perform an additional accept.

3. Uppercase Logic and Buffer Management

- The process_echo function operates on the existing buffer (**In-place**). There is no new memory allocation (malloc) and no unnecessary copies (memcpy). This is the level of efficiency expected in System Programming.

4. Handling Zombie Processes

In Linux, when a child process ends, it remains in the process table as a **"Zombie"** until the parent calls wait(). If the server runs for a long time without waitpid, the process table will fill up. We therefore added the line waitpid(-1, NULL, WNOHANG) to clean up terminated processes in a non-blocking manner.